

JUSTIFICACIÓN USO DE IA

- **Requerimiento 1 (en logic.py)**

Descripción por función: Se utilizó IA para dar una guía general y ayudar a estructurar mejor el problema para luego empezar a codificar. Además, se fue comprobando constantemente por medio de la IA si se cometía algún error (como paréntesis sin cerrar o minúsculas mal puestas).

Código:

```
def req_1(catalog, nom_producto):  
    Retorna el resultado del requerimiento 1  
    """  
    Tiempo_inicial = get_time()  
  
    catalog = catalog["array"]  
  
    n = al.size(catalog)  
  
    contador = 0  
    sum_price = 0; min_price = None; max_price = None  
    sum_discount = 0; min_discount = None; max_discount = None  
    sum_boxes = 0; min_boxes = None; max_boxes = None  
    sum_marketing = 0; min_marketing = None; max_marketing = None  
    conteo_años = {}  
    max_amount = None  
    ubicacion_max_amount = None  
    less_amount = None  
    ubicacion_less_amount = None  
  
    for i in range (0, n):  
  
        fila = al.get_element(catalog, i)  
  
        if nom_producto == fila["Product"]:  
  
            contador+=1  
  
            price = fila["Price_per_Box"]  
            sum_price += price  
            if min_price == None:  
                min_price = price  
            else:  
                min_price = min(min_price, price)  
  
            if max_price == None:  
                max_price = price  
            else:  
                max_price = max(max_price, price)  
  
            discount = fila["Discount_Pct"]  
            sum_discount+= discount  
            if min_discount == None:  
                min_discount = discount  
            else:  
                min_discount = min(min_discount,discount)  
  
            if max_discount == None:  
                max_discount = discount  
            else:  
                max_discount = max(max_discount,discount)
```

```

boxes = fila["Boxes_Shipped"]
sum_boxes+= boxes
if min_boxes == None:
    min_boxes = boxes
else:
    min_boxes = min(min_boxes,boxes)

if max_boxes == None:
    max_boxes = boxes
else:
    max_boxes = max(max_boxes,boxes)

marketing = fila["Marketing_Spend"]
sum_marketing+= marketing
if min_marketing == None:
    min_marketing = marketing
else:
    min_marketing = min(min_marketing,marketing)

if max_marketing == None:
    max_marketing = marketing
else:
    max_marketing = max(max_marketing,marketing)

fecha = fila["Order_Date"]
año = fecha[:4]
if año in conteo_años:
    conteo_años[año] = conteo_años[año] + 1
else:
    conteo_años[año] = 1

amount = fila["Amount"]
if (max_amount == None) or (amount > max_amount):
    max_amount = amount
    ubicacion_max_amount = i

if (less_amount == None) or (amount < less_amount):
    less_amount = amount
    ubicacion_less_amount = i

if contador == 0:
    return "No existe ese nombre en los datos"

año_top = None
max_conteo = 0

```

```

for año in conteo_años:
    if conteo_años[año] > max_conteo:
        max_conteo = conteo_años[año]
        año_top = año

fila_mayor = al.get_element(catalog, ubicacion_max_amount)
a = fila_mayor["Order_ID"]
b = fila_mayor["Country"]
c = fila_mayor["Order_Date"]
d = fila_mayor["Price_per_Box"]

info_mayor_amount = "Mayor amount = "+str(a)+" "+str(b)+" "+str(c)+" "+str(d)+" "+str(max_amount)

fila_menor = al.get_element(catalog, ubicacion_less_amount)
e = fila_menor["Order_ID"]
f = fila_menor["Country"]
g = fila_menor["Order_Date"]
h = fila_menor["Price_per_Box"]

info_menor_amount = "Menor amount = "+str(e)+" "+str(f)+" "+str(g)+" "+str(h)

prom_price = sum_price / contador
prom_discount = sum_discount / contador
prom_boxes = sum_boxes / contador
prom_marketing = sum_marketing / contador

Tiempo_final = get_time()
Tiempo_total = delta_time(Tiempo_inicial, Tiempo_final)

return Tiempo_total, contador, prom_price, max_price, min_price, prom_discount, max_discount, min_discount, prom_boxes, max_boxes, min

```

Justificación: Al pedir tantas cosas en una sola función, llegué a acomplejarme demasiado al punto en el que mi código ya no tenía sentido y no sabía cómo estructurar todo lo que debía hacer. Además, esto mismo me distraía y me hacía cometer errores en lo poco que ya tenía.

Evidencia:



Ahora necesito hacer este requerimiento, dame una base o ideas para empezar a desarrollarlo

Leyó y agregado a memoria >

[Probable] Antes de darte una estructura, revisa una suposición peligrosa: si estás pensando en filtrar primero (una pasada creando una sublista con los pedidos del producto) y luego calcular promedios/mín/máx en otra pasada, eso es justo lo que no te puedes permitir aquí. El requerimiento pide medir tiempo de ejecución, y sobre una single linked list cada recorrido completo es $O(n)$ — si haces 3 o 4 recorridos en vez de uno solo, tu tiempo se dispara sin necesidad real.

[Probable] La forma correcta es un único recorrido con acumuladores, comparando cada

Conocimiento obtenido:

- Aprendí a hacer mejor mis recorridos en la estructura de datos, a como guardar varias variables en una sola línea de código y a organizar mejor las asignaciones y recorridos para cada problema.

- **Requerimiento 2 (en logic.py)**

Descripción por función: Usé la inteligencia artificial como guía de apoyo en la función req_2 del archivo logic para entender el enunciado, comprobar que iba por buen camino y qué errores se podían corregir y/o simplificar para poder entender mejor la implementación y mejorar el orden de complejidad.

Código:

```
def req_2(precio_minimo, precio_maximo, catalog):
    """
    Retorna el resultado del requerimiento 2
    """
    start_time = get_time()
    cantidad_pedidos = 0
    suma_discount_pct = 0
    suma_marketing_spend = 0
    suma_prices_per_box = 0

    pedido_mas_reciente = None
    pedido_menor_amount = None
    pedido_mayor_amount = None

    catalog = catalog["array"]
    size = al.size(catalog)

    for i in range(size):
        fila = al.get_element(catalog, i)
        precio = fila["Price_per_Box"]

        if precio_minimo <= precio <= precio_maximo:
            cantidad_pedidos += 1
            suma_prices_per_box += precio
            suma_discount_pct += fila["Discount_Pct"]
            suma_marketing_spend += fila["Marketing_Spend"]

            if pedido_mas_reciente is None or fila["Order_Date"] >
                pedido_mas_reciente["Order_Date"]:
                pedido_mas_reciente = fila
```

```

        elif fila["Order_Date"] ==
pedido_mas_reciente["Order_Date"] and fila["Amount"] >
pedido_mas_reciente["Amount"]:
            pedido_mas_reciente = fila

        if pedido_menor_amount is None or fila["Amount"] <
pedido_menor_amount["Amount"]:
            pedido_menor_amount = fila
            elif fila["Amount"] == pedido_menor_amount["Amount"]
and precio < pedido_menor_amount["Price_per_Box"]:
                pedido_menor_amount = fila

        if pedido_mayor_amount is None or fila["Amount"] >
pedido_mayor_amount["Amount"]:
            pedido_mayor_amount = fila
            elif fila["Amount"] == pedido_mayor_amount["Amount"]
and precio < pedido_mayor_amount["Price_per_Box"]:
                pedido_mayor_amount = fila

    promedio_discount_pct = suma_discount_pct / cantidad_pedidos
if cantidad_pedidos > 0 else 0
    promedio_marketing_spend = suma_marketing_spend /
cantidad_pedidos if cantidad_pedidos > 0 else 0
    promedio_prices_per_box = suma_prices_per_box /
cantidad_pedidos if cantidad_pedidos > 0 else 0

    end_time = get_time()
    tiempo_ejecucion = delta_time(start_time, end_time)

    return (tiempo_ejecucion, cantidad_pedidos,
promedio_discount_pct, promedio_marketing_spend,
promedio_prices_per_box, pedido_mas_reciente,
pedido_mayor_amount, pedido_menor_amount)

```

Justificación: Me seguían apareciendo errores y me atasqué intentando resolver el problema por mi cuenta después de intentarlo unas cuantas veces.

Evidencia:

Necesito tu ayuda y te voy a contextualizar primero con todo lo necesario. Tengo este proyecto de estructura de datos y algoritmos, te adjunto guía completa (pdf), datos (csv), y lo que tengo para que entiendas la estructura (logic y view sin resolver), además te voy a adjuntar el laboratorio pasado (logic y view resuelto todo) como para que entiendas mas o menos la manera de trabajar (con su respectivas correcciones que toca hacer (rubrica calificada)) y la rubrica del proyecto.

Necesito que por cada duda que tenga en este chat me expliques de forma fácil de entender y completa, qué hace cada cosa y cual es el paso a paso, todo con ejemplos extra si es posible y demás cosas importantes para yo comprender mejor el requerimiento como tal y las implementaciones hechas. Cabe resaltar que no tengo tanto nivel de programación entonces no te inventes cosas complejas o difíciles, solo haz las cosas como el laboratorio y el pdf digan.

Memoria leída, ejecutó un comando, memoria actualizada >

Ya revisé todo lo que compartiste: la guía completa del Reto 1 (ventas de chocolate), el CSV de datos, tu `logic.py` / `view.py` sin resolver, el laboratorio anterior resuelto (catálogo de libros) y la rúbrica con los comentarios del profesor. Con eso ya tengo el panorama completo:

El Reto 1 en resumen:



de forma corta y rapida, en este requerimiento me piden el pedido con menor amount o mayor amount o ambas, no entiendo

Te piden **ambas cosas**: el pedido de menor Amount y el pedido de mayor Amount, dentro del rango de precio que estás filtrando (Req. 2).

Es decir, de todos los pedidos que caen en ese rango de `Price_per_Box`, tienes que encontrar:

- El que tiene el **Amount más bajo** de todos (con sus 6 datos: Product, Country, Channel, Order_Date, Price_per_Box, Amount).
- El que tiene el **Amount más alto** de todos (con esos mismos 6 datos).

Son dos pedidos distintos que debes reportar por separado (a menos que casualmente el más barato y el más caro coincidan, lo cual sería raro).



```
def req_2(precio_minimo,
precio_maximo,catalog):
''' Retorna el resultado d
el requerimiento 2 ''' # T
ODO: Modificar el requeri
miento 2 start_time = g...
```

PASTED

intenté por mi propia cuenta hacer el requerimiento 2 de logic y esto fue lo que salió, necesito que me expliques de forma precisa y fácil de entender los errores que tuve y cosas que se pueden mejorar según toda la información de este chat

Pensó durante 1m 7s Marcando el riesgo de usar listas nativas en vez del TAD. >

Revisé tu código con calma. Antes de entrar en detalle: la buena noticia es que la **estructura general está bien pensada** (calculas promedios, buscas el más reciente, buscas el de mayor/menor Amount). El problema está en varios detalles concretos. Te los explico uno por uno, del más importante al menos importante.

● **Error #1 (el más grave): confundes "el valor" con "la posición"**

Esto es la raíz de casi todos los demás errores. Míste este fragmento:

Conocimiento Obtenido:

- Aprendí que no debo confundir índices con un valor a la hora de recorrer listas porque luego al compararlos se generará un error, además de que también puedo facilitarme la vida si en logic guardo toda la información del pedido y en view printeo lo que necesiten de ese pedido en vez de guardar todo en miles de variables en logic y printearlas en view.

- **Requerimiento 4 (en logic. py)**

Descripción por función: Se utilizó IA para acortar lo más posible el tiempo de ejecución debido a que el programa no corría. Además, se utilizó para pedir ideas para almacenar los dos datos de mayor amount.

Código:

```
def req_4(catalog, producto, pais):

    # TODO: Modificar el requerimiento 4
    inicio = get_time()

    catalog = catalog["single_linked"]

    total_pedidos = 0
    prom_price = 0
    prom_discount = 0
    prom_marketing = 0
    prom_boxes = 0

    top_amount_1 = None
    top_amount_2 = None

    nodo_actual = catalog["first"]

    while nodo_actual is not None:
        fila = nodo_actual["info"]

        if (fila["Product"] == producto) and (fila["Country"] == pais):

            total_pedidos += 1

            price = float(fila["Price_per_Box"])
            discount = float(fila["Discount_Pct"])
            marketing = float(fila["Marketing_Spend"])
            boxes = float(fila["Boxes_Shipped"])

            prom_price += price
            prom_discount += discount
            prom_marketing += marketing
            prom_boxes += boxes

            amount = float(fila["Amount"])
            order_id = fila["Order_ID"]
            channel = fila["Channel"]
            fecha = fila["Order_Date"]

            posible = {"amount": amount,
                       "marketing": marketing,
                       "order_id": order_id,
                       "channel": channel,
                       "fecha": fecha,
                       "boxes": boxes
                      }

            if top_amount_1 is None:
                top_amount_1 = posible
```



```

elif (amount > top_amount_1["amount"]) or ((amount == top_amount_1["amount"]) and (marketing < top_amount_1["marketing"])) or
    top_amount_2 = top_amount_1
    top_amount_1 = posible

elif top_amount_2 is None:
    top_amount_2 = posible

elif (amount > top_amount_2["amount"]) or ((amount == top_amount_2["amount"]) and (marketing < top_amount_2["marketing"])) or
    top_amount_2 = posible

nodo_actual = nodo_actual["next"]

if total_pedidos == 0:
    return "No hubo pedidos con esa combinación"

prom_price = prom_price / total_pedidos
prom_discount = prom_discount / total_pedidos
prom_marketing = prom_marketing / total_pedidos
prom_boxes = prom_boxes / total_pedidos

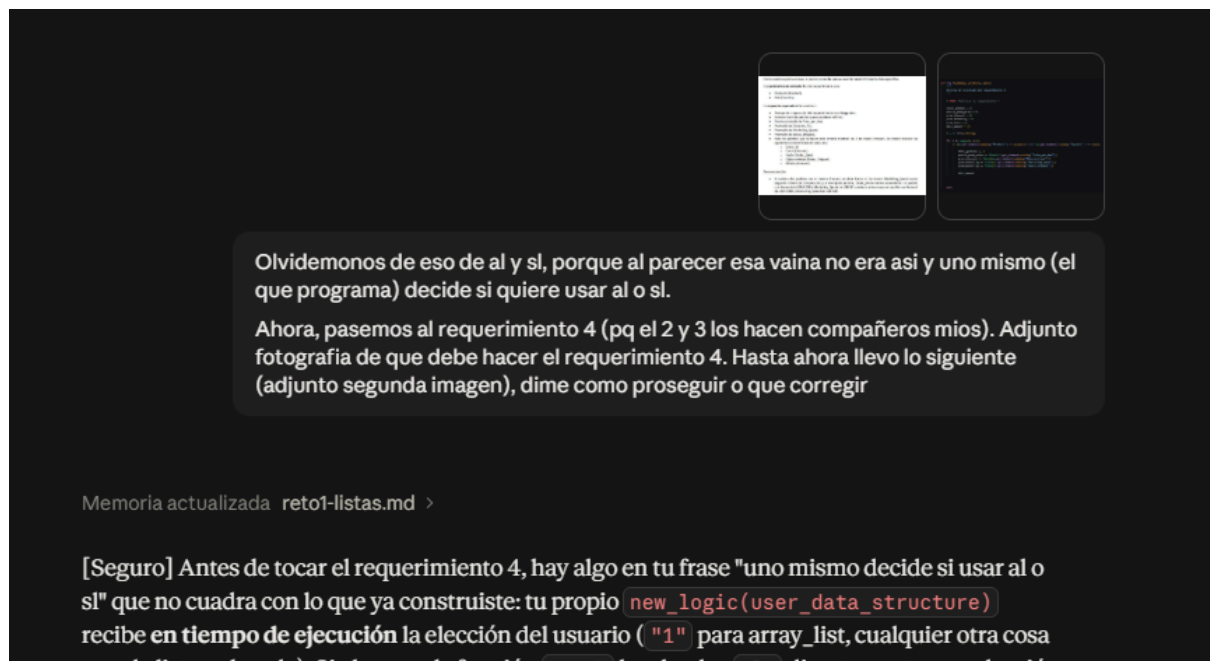
final = get_time()
tiempo_total = delta_time(inicio, final)

return tiempo_total, prom_price, prom_discount, prom_marketing, prom_boxes, top_amount_1, top_amount_2

```

Justificación: El programa al ejecutarse no corría, por lo que tuve que buscar otras maneras de que la complejidad temporal fuera menor. Además, no se me ocurrían ideas para poder almacenar ambos datos de amount, por lo que recurrí para que me diera posibles ideas y me corrigiera la idea que ya tenía (un diccionario).

Evidencia:



Olvidemonos de eso de al y si, porque al parecer esa vaina no era así y uno mismo (el que programa) decide si quiere usar al o si.

Ahora, pasemos al requerimiento 4 (pq el 2 y 3 los hacen compañeros míos). Adjunto fotografía de que debe hacer el requerimiento 4. Hasta ahora llevo lo siguiente (adjunto segunda imagen), dime como proseguir o que corregir

Memoria actualizada reto1-listas.md >

[Seguro] Antes de tocar el requerimiento 4, hay algo en tu frase "uno mismo decide si usar al o si" que no cuadra con lo que ya construiste: tu propio `new_logic(user_data_structure)` recibe en tiempo de ejecución la elección del usuario ("`1`" para `array_list`, cualquier otra cosa para la lista enlazada). Si ahora cada función `new_logic` y `new_logic` directamente con elección

Conocimiento obtenido:

- Aprendí cómo funcionaban los nodos y como avanzaban. También me ayudo a recordar como podía almacenar varios datos por medio de diccionarios donde simplemente podía guardar un dato viejo y añadir el nuevo.

• Load Data (en view.py y en logic.py)

Descripción por función: Utilizamos la IA como guía de apoyo y corrector para solucionar la carga de datos puesto que es la base de las pruebas de todo el reto y nada que corrían.

Código:

```
def load_data(catalog, filename):
    """
    Carga los datos del reto
    """
    start_time = get_time()

    with open(data_dir + filename, encoding='utf-8-sig') as f:
        archivo = csv.DictReader(f)

        pedido_menor = None
        pedido_mayor = None

        for fila in archivo:
            fila["Discount_Pct"] = float(fila["Discount_Pct"])
            fila["Price_per_Box"] = float(fila["Price_per_Box"])
            fila["Marketing_Spend"] = float(fila["Marketing_Spend"])
            fila["Boxes_Shipped"] = int(fila["Boxes_Shipped"])
            fila["Amount"] = float(fila["Amount"])

            al.add_last(catalog["array"], fila)
            sl.add_last(catalog["single_linked"], fila)

            if pedido_menor is None or fila["Amount"] <
pedido_menor["Amount"]:
                pedido_menor = fila
                elif fila["Amount"] == pedido_menor["Amount"] and
fila["Price_per_Box"] < pedido_menor["Price_per_Box"]:
                    pedido_menor = fila

            if pedido_mayor is None or fila["Amount"] >
pedido_mayor["Amount"]:
                pedido_mayor = fila
                elif fila["Amount"] == pedido_mayor["Amount"] and
fila["Price_per_Box"] < pedido_mayor["Price_per_Box"]:
                    pedido_mayor = fila

        total_pedidos = al.size(catalog["array"])

        primeros_5 = al.new_list()
        for i in range(min(5, total_pedidos)):
            al.add_last(primeros_5, al.get_element(catalog["array"], i))
```

```

ultimos_5 = al.new_list()
for i in range(max(0, total_pedidos - 5), total_pedidos):
    al.add_last(ultimos_5, al.get_element(catalog["array"], i))

end_time = get_time()
tiempo_carga = delta_time(start_time, end_time)

return tiempo_carga, total_pedidos, pedido_menor, pedido_mayor,
primeros_5, ultimos_5

```

Justificación: No paraban de salir errores a la hora de cargar datos, y no podíamos probar ningún requerimiento, aparte de que no habíamos entendido bien el enunciado por lo cual nos llevó varios intentos de prueba y error para lograr hacer que funcione. Sin mencionar que intentamos usar el tabulate en view y fue todo un lío.

Evidencia:

tengo que cargar datos ahora, solo llevo esto que me falta para cumplir con lo que dice el pdf

```

def load_data(catalog, filename):
    """
    Carga los datos del reto
    """
    # TODO: Realizar la carga de datos
    start_time = get_time()

```

Mostrar más

hace 8 horas   

Memoria recuperada >

Vas por buen camino con la estructura del `for`, pero hay varios puntos importantes que hay que resolver antes de seguir. Te los explico:

● **Error #1 (el más peligroso, porque no se ve a simple vista):** todos los datos llegan como texto

000 000 000

PDF

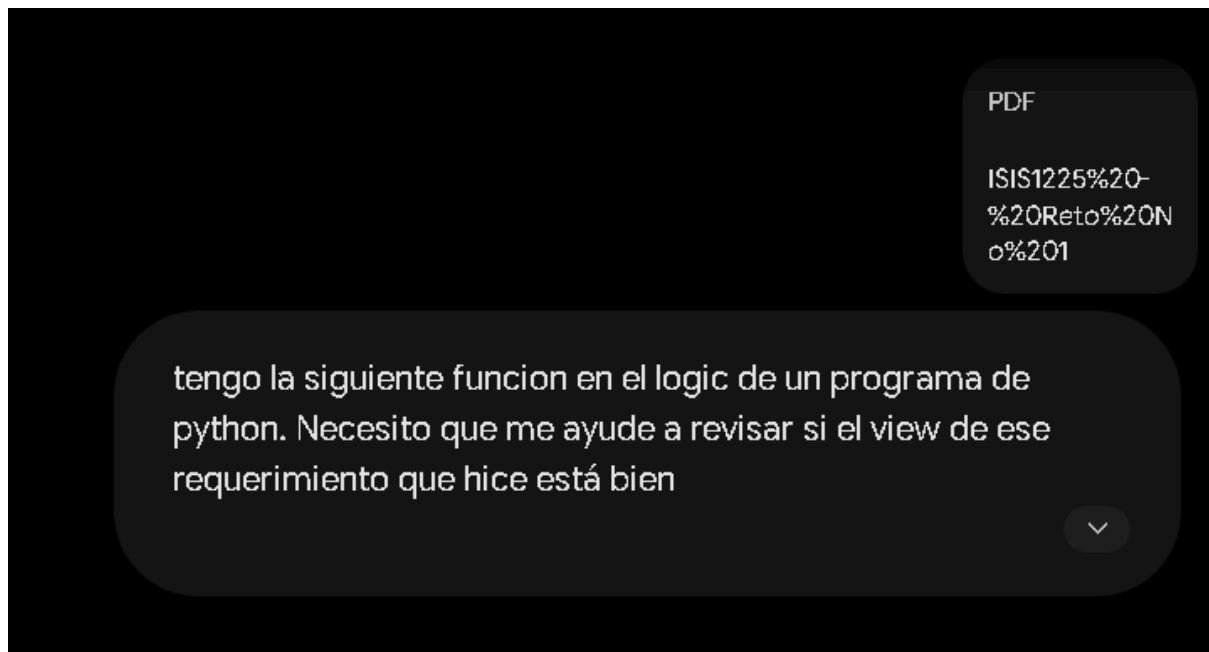
al), el `view` necesita recorrerlos usando las operaciones de esa estructura (`al.size()` y `al.get_element()`).

A:\Local\Python\python.exe

```
n (module)
t, in (modules)
```

PDF

Para solucionarlo, ve a esa misma terminal de PowerShell donde te salió el error y ejecuta el siguiente comando:



Conocimiento Obtenido:

- Gracias a este apoyo pude recordar a cómo leer un csv y cómo poder procesarlo, también a cómo implementar listas encadenadas o arreglos para poder ser procesados como tal y que los requerimientos las puedan usar y también aprendí el uso correcto de funciones un poco raras como “os” o el encoding de un csv. Se nos complicó además las funciones de los primeros 5 y últimos 5 (más que todo acerca de cómo recorrerlas), entonces la IA aparte de despejar dudas, también nos ayudó a revisar si íbamos por buen camino cada vez que hacíamos un cambio.

1. Descripción por función del uso de IAG

- **Función req_3:** Se utilizó IA (Gemini) para asistir en la creación de una estructura del código pasando de esquemas de apuntes en papel a pseudocódigo con el que se trabajó para llegar a una implementación que posteriormente se verificó que no tuviera errores de sintaxis y que la estructura funcionara
- **Función req_6:** Se utilizó IA (Gemini) para ayudar con la estructura que guarda los canales Fechas, agrupación por canales, cálculo de mínimos/máximos y promedios) además de para corregir y revisar la sintaxis debido a un error al momento de correr el código donde no leía los datos .

2. Justificación del uso

Se usó la IA ya que necesitan de un apoyo conceptual para refrescar el conocimiento sobre el uso de Python además de verificar la correcta sintaxis e indentación de las funciones. Con el uso de la IA se quería lograr comprender cómo modelar agrupaciones de datos y realizar los conteos de frecuencia. Y para corregir un error al correr que no se leían los código se le pidió a la ia que los modificara.

3.

```
Ingrese la Fecha de Inicio (YYYY-MM-DD): 2022-01-22
Ingrese la Fecha de Fin (YYYY-MM-DD): 2023-07-26
Traceback (most recent call last):
  File "c:\Users\Fede\Documents\Nicolas
home\2026_2\EDA\Reto1-G01\main.py", line 11, in <module>
    main()
  File "c:\Users\Fede\Documents\Nicolas
home\2026_2\EDA\Reto1-G01\main.py", line 6, in main
    view.main()
  File "c:\Users\Fede\Documents\Nicolas
home\2026_2\EDA\Reto1-G01\App\view.py", line 316, in main
    print_req_6(control)
  File "c:\Users\Fede\Documents\Nicolas
home\2026_2\EDA\Reto1-G01\App\view.py", line 247, in
print_req_6
    resultados = control.req_6(start_date, end_date)
                  ^^^^^^^^^^^^^
AttributeError: 'dict' object has no attribute 'req_6'
```

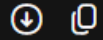
Ese es exactamente el error del que te hablaba en el mensaje anterior. El problema ocurre porque en tu archivo `view.py` todavía tienes la línea escrita con la estructura antigua.

El error indica que en la línea 247 de `view.py` tienes esto:

Ese es exactamente el error del que te hablaba en el mensaje anterior. El problema ocurre porque en tu archivo `view.py` todavía tienes la línea escrita con la estructura antigua.

El error indica que en la línea 247 de `view.py` tienes esto:

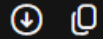
Python



```
resultados = control.req_6(start_date, end_date)
```

Para solucionarlo, debes cambiar esa línea para llamar a la función a través del módulo `logic` y pasarle `control` como el primer parámetro. Cámbiala por esta:

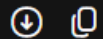
Python



```
resultados = logic.req_6(control, start_date, end_date)
```

Asegúrate también de revisar la función `print_req_3` en tu archivo `view.py`, ya que tenía exactamente el mismo problema. Debe verse así:

Python



```
resultados = logic.req_3(control, pais, canal)
```

```
Ingrese el País a buscar (ej. United States): Australia
Ingrese el Canal a buscar (ej. Online): Retail
Traceback (most recent call last):
  File "c:\Users\Fede\Documents\Nicolas
home\2026_2\EDA\Reto1-G01\main.py", line 11, in <module>
    main()
  File "c:\Users\Fede\Documents\Nicolas
home\2026_2\EDA\Reto1-G01\main.py", line 6, in main
    view.main()
  File "c:\Users\Fede\Documents\Nicolas
home\2026_2\EDA\Reto1-G01\App\view.py", line 317, in main
    print_req_3(control)
  File "c:\Users\Fede\Documents\Nicolas
home\2026_2\EDA\Reto1-G01\App\view.py", line 168, in
print_req_3
    resultados = logic.req_3(control, pais, canal)
    ~~~~~^~~~~~

  File "c:\Users\Fede\Documents\Nicolas
home\2026_2\EDA\Reto1-G01\App\logic.py", line 270, in req_3
    catalog=catalog["Array"]
    ~~~~~^~~~~~

KeyError: 'Array'
```



El error `KeyError: 'Array'` significa que estás intentando acceder a una llave (key) llamada `"Array"` dentro del diccionario `catalog` (que es la variable `control` que enviaste desde la vista), pero esa llave **no existe** en el diccionario.

Python



```
catalog = catalog["Array"]
```

En este tipo de proyectos de Estructuras de Datos, es común que la estructura principal sea un diccionario, pero el nombre de la llave donde realmente se guardan los datos suele ser otro (por ejemplo, "elements", "elementos", "data", o "pedidos").

¿Cómo solucionarlo?

1. **Averigua el nombre correcto de la llave:**

Ve a tu archivo `logic.py`, y justo una línea antes de la 270, agrega temporalmente un `print` para ver qué contiene el diccionario:

Python



```
print("Llaves disponibles en catalog:", catalog.keys())  
catalog = catalog["Array"] # La línea que está fallando
```

2. **Ejecuta el programa de nuevo:**

Haz el mismo proceso. Antes de que salga el error en rojo, verás un texto impreso en la terminal que dirá algo como:

Llaves disponibles en catalog: dict_keys(['elements', 'size']) o algo muy similar.

dame la version del code con sl en vez de al



Para usar `sl` (Single Linked List / Lista Enlazada Simple) en lugar de recorrerla manualmente con el bucle `while`, puedes extraer los datos usando las funciones de la librería de listas enlazadas (`sl.size()` y `sl.get_element()`), exactamente de la misma manera que lo estructuraste para el Requerimiento 5.

Aquí tienes la versión completa de `req_3` utilizando las funciones de `sl`:

Python



```
def req_3(catalog, Country, Channel):  
    """  
    Retorna el resultado del requerimiento 3 usando sl (Single Linked List)  
    """  
    start_time = get_time()  
  
    # 1. Acceder a la lista enlazada  
    catalog = catalog["single_linked"]  
  
    # 2. Obtener el tamaño usando la librería sl  
    tamaño = sl.size(catalog)  
  
    N = 0  
    suma_precio = 0
```

```

def req_3(catalog, Country, Channel):
    """
    Retorna el resultado del requerimiento 3 usando sl (Single Linked List)
    """
    start_time = get_time()

    # 1. Acceder a la lista enlazada
    catalog = catalog["single_linked"]

    # 2. Obtener el tamaño usando la librería sl
    tamaño = sl.size(catalog)

    N = 0
    suma_precio = 0
    suma_descuento = 0
    suma_marketing = 0
    suma_cajas = 0

    conteo_productos = {}
    conteo_anios = {}

    # 3. Recorrer usando sl.get_element(catalog, i)
    for i in range(tamaño):
        fila = sl.get_element(catalog, i)

        # Filtrar por País y Canal
        if fila["Country"] == Country and fila["Channel"] == Channel:
            N += 1
            suma_precio += fila["Price_per_Box"]
            suma_descuento += fila["Discount_Pct"]
            suma_marketing += fila["Marketing_Spend"]
            suma_cajas += fila["Boxes_Shipped"]

            # Conteo para el producto más frecuente
            prod = fila["Product"]
            conteo_productos[prod] = conteo_productos.get(prod, 0) + 1

            # Conteo para el año con más pedidos
            fecha = fila["Order_Date"]
            anio = str(fecha).split("-")[0] if fecha else "Desconocido"
            conteo_anios[anio] = conteo_anios.get(anio, 0) + 1

    # 4. Calcular promedios y modas
    if N > 0:
        p_precio = suma_precio / N
        p_desc = suma_descuento / N
        p_mkt = suma_marketing / N
        p_cajas = suma_cajas / N

        moda_prod = max(conteo_productos, key=conteo_productos.get) if conteo_productos else "Desconocido"
        moda_anio = max(conteo_anios, key=conteo_anios.get) if conteo_anios else "Desconocido"
    else:
        p_precio = 0
        p_desc = 0
        p_mkt = 0
        p_cajas = 0
        moda_prod = "Desconocido"
        moda_anio = "Desconocido"

```

4. Aprender a depurar este tipo de proyectos de estructuras de datos exige prestar máxima atención a las convenciones de nomenclatura, como las minúsculas exactas en las llaves de los contenedores (array o single_linked), y a la naturaleza de cada estructura para utilizar

correctamente los métodos de navegación (como `sl.get_element()` en lugar de accesos directos por clave). Asimismo, interpretar con precisión las trazas de error y asegurar el almacenamiento físico de los archivos evita discrepancias de ejecución en la terminal, mientras que conocer los límites reales del dataset resulta fundamental al aplicar filtros por rangos, como las fechas, para garantizar resultados válidos y evitar consultas vacías.